

08 — MODERN C++ FEATURE MAP (C++11 → C++23)

Interviewers ask "what's your favorite C++17 feature" or "how would you do X in modern C++". This is the per-standard cheat sheet. Depth on the marquee items lives in the topic files; this is the map + the one-liners.

How to think about this. Don't memorize feature lists — memorize the *arc*. C++ has been steadily moving decisions from runtime to compile time (constexpr everything, concepts, `if constexpr`), replacing raw ownership with typed ownership (smart pointers, `string_view` / `span`), and giving errors first-class values instead of hidden control flow (`optional`, `expected`). When asked "how would you do X in modern C++," reach for the tool that moves the check earlier, makes ownership explicit, or turns a failure into a value.

Take-home lesson. Each standard solved a class of problem, not a grab-bag of features. C++11 = *ownership & move*. C++14 = *polish the lambda/constexpr story*. C++17 = *vocabulary types & compile-time branching*. C++20 = *constraints, ranges, coroutines*. C++23 = *ergonomics & `expected`*. Framing your answer around the *problem* each feature solved reads as senior.

C++11 — THE BIG BANG

- Move semantics & rvalue references (`&&`, `std::move`, `std::forward`).
- `auto`, `decltype`, trailing return types.
- Lambdas.
- Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`); `make_shared`.
- `nullptr`, `enum class`, `constexpr`, `override` / `final`.
- Uniform init `{}`, `initializer_list`.

- Range-based for.
- Variadic templates, alias templates (`using`).
- Threading: `<thread>`, `<mutex>`, `<atomic>`, `<future>`, memory model.
- `std::function`, `std::bind`, `std::tuple`, `std::array`, unordered containers.
- `noexcept`, defaulted/deleted functions (`= default` / `= delete`), delegating & inheriting constructors.
- `static_assert`, thread-safe magic statics.

C++14 — THE POLISH RELEASE

- Generic lambdas (`[] (auto x) {}`), init-capture (`[x=std::move(y)]`).
- Return type deduction for normal functions (`auto f() { ... }`).
- Variable templates, `std::make_unique` (missing in 11).
- Relaxed `constexpr` (loops, locals).
- `std::integer_sequence`, digit separators `1'000'000`.

C++17 — VERY INTERVIEW-RELEVANT

- Structured bindings `auto [a,b] = ...`.
- `if constexpr` — compile-time branching (kills much SFINAE).
- `std::optional`, `std::variant`, `std::any`, `std::string_view`.
- CTAD (class template arg deduction), deduction guides.
- Fold expressions `(... + args)`.
- Guaranteed copy elision.
- `inline` variables; nested namespaces `namespace a::b::c`.
- `std::filesystem`, parallel STL algorithms (`std::execution::par`).
- `[[nodiscard]]`, `[[maybe_unused]]`, `[[fallthrough]]`.

- `std::scoped_lock`, `std::shared_mutex`, `std::apply`, `std::invoke`.

C++20 — THE SECOND BIG BANG

- Concepts & `requires` — constrained templates, better errors.
- Ranges — composable lazy views with `|`.
- Coroutines — `co_await` / `co_yield` / `co_return` (low-level; libs build on it).
- Modules — `import` / `export` (replacing headers; toolchain-dependent).
- `<=>` spaceship operator — auto-generate comparison operators.
- `std::span`, `std::jthread` + `stop_token`, `std::atomic_ref`, `std::atomic<shared_ptr>`.
- `constexpr`, `constinit`, `constexpr` almost everywhere (even `new` / `vector` / `string` in constant evaluation).
- Designated initializers `{.x=1, .y=2}`; `[[likely]]` / `[[unlikely]]`; `std::format`; calendar/timezone in `<chrono>`; `std::bit_cast`.

C++23 — THE REFINEMENTS

- `std::expected<T,E>` — value-or-error without exceptions.
 - `std::mdspan` — multi-dimensional array view.
 - `std::print` / `std::println` — finally, ergonomic formatted output.
 - `std::flat_map` / `flat_set` — sorted-vector-backed (cache-friendly).
 - `std::generator` — coroutine-based lazy ranges.
 - Deducing `this` (explicit object parameter) — dedupes const/non-const & CRTP.
 - `if constexpr`, `[[assume]]`, monadic ops on `optional` (`and_then` / `transform` / `or_else`), ranges additions (`zip`, `enumerate`, `to`).
-

THE VOCABULARY TYPES (KNOW WHEN TO PICK EACH)

Type	Use
<code>optional<T></code>	maybe-a-value; nullable return without sentinels
<code>variant<Ts...></code>	type-safe tagged union; visit with <code>std::visit</code>
<code>any</code>	type-erased single value; rarely the right tool
<code>expected<T,E></code> (23)	value or error; exception-free error propagation
<code>tuple<Ts...></code>	heterogeneous fixed collection; structured-bind it
<code>string_view</code> / <code>span</code>	non-owning read views; cheap params, mind lifetime

`string_view` / `span` borrow, they don't own. Each is just a pointer + length into someone else's buffer — it keeps nothing alive. That makes them ideal as function parameters (the argument outlives the call) and dangerous as members or return values (the underlying storage may die first, leaving a dangling view). Use them to *pass* data cheaply, never to *own* or *outlive* it. (See File 01's dangling-`string_view` Stop-and-think.)

`std::visit` over a `variant` + overloaded lambdas is the idiomatic

"sum type / pattern match":

```
std::visit(overloaded{
    [](int i){ /*...*/ },
    [](const std::string& s){ /*...*/ }
}, v);
```

The `overloaded` helper itself is a tidy C++17 idiom worth being able to write:

```
template <class... Ts> struct overloaded : Ts... { using
Ts::operator()...; };
template <class... Ts> overloaded(Ts...) -> overloaded<Ts...>; // CTAD
guide
```

Structured bindings make map/`variant` iteration clean:

```
for (const auto& [key, value] : some_map) { use(key, value); }    // C++17
if (auto [it, inserted] = m.try_emplace(k, v); inserted) { /* new */ }
```

Stop and think. When would you pick `variant<A,B>` over a base class with two derived types?

► answer

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- Name one marquee feature from each of C++11/14/17/20/23.
- When do you pick `optional` vs `variant` vs `expected`?
- What does `<=>` generate?

Level 2 — Apply

1. Rewrite a factory returning error codes to use `optional`, then `expected`.
2. Replace a hand-written tagged union with `variant` + `visit` (write the `overloaded` helper).
3. Show `<=>` generating all six relational operators for a struct.

Level 3 — Transfer

4. Use `string_view` to eliminate copies in a parsing function; then point to the exact line where a lifetime bug could hide.
5. Take a header-only utility and give the "how would this be a module" answer — what changes, what improves.
6. For one feature from each standard, state the *problem it solved*, not just what it does.

Level 4 — Teach

- Pitch modern C++ to someone stuck on C++98: pick the three features that most change how they'd write code, and justify each by the problem it removes.

CONNECTS TO

- File 02 — move semantics (C++11) is the foundation the whole "modern" era is built on.
- File 03 — concepts, `if constexpr`, fold expressions, CTAD are the template-era modernizations.
- File 04 — ranges, `span`, `string_view`, `flat_map` modernize container/algorithm use.
- File 07 — `optional`/`expected` change how you model errors vs exceptions; `inline` variables, structured bindings.